Tema 6

Almacenamiento de información

Lenguaje de marcas y sistemas de gestión de la información

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 3
2. Sistemas de almacenamiento de información 4
2.1 Ficheros 4
2.2 Bases de datos XML enabled 4
2.3 Bases de datos nativas 5
3. Manipulación de información en documentos XML: inserción y extracción 7
4. Técnicas de búsqueda de información en documentos XML 8
5. Lenguaje de consulta y manipulación 9
5.1 XPath 9
5.1.1 Los nodos en XPath 11
5.1.2 Como seleccionar los nodos de un documento 12
5.1.3 Utilizando predicados para mejorar la búsqueda 14
5.1.4 Cambiar varias rutas 16
5.1.5 Comodines 17
5.2 XQuery 19
5.2.1 Aplicación para trabajar con XQuery 25
6. Almacenamiento XML nativo 27
6.1. Instalación y ejecución de una base de datos XML nativa en eXist-db 28
7. Herramientas de tratamiento y almacenamiento de información en formato XML 31
8. Resumen 32
Glosario 33
Bibliografía 36

Objetivos de la unidad

- Identificación de los principales métodos de almacenamiento de la información usada en documentos XML.
- Selección de tecnologías de almacenamiento de información eficientes basadas en sus características distintivas.
- Utilización de sistemas de gestores de datos relacionales en el almacenamiento de información en formato XML.
- Identificación de las características de los sistemas gestores de bases de datos nativas XML.

1. Introducción

Una **base de datos XML** permite almacenar datos en **formato XML**. Al igual que cualquier base de datos, este sistema permite acceder a los datos almacenados y modificarlos según sea necesario. La adopción de XML como formato de intercambio de información ha generado la necesidad de utilizar este tipo de bases de datos.

Las bases de datos XML son una buena solución cuando la información por almacenar tiene estructura jerárquica. Puede considerarse un documento XML como una base de datos, puesto que almacena información y tiene una estructura determinada. Puede ser consultado mediante lenguajes como **XQuery** y **XPath**.

2. Sistemas de almacenamiento de información

Existen tres formas de almacenar información XML:

- En ficheros.
- En bases de datos XML no nativas llamadas también XML enabled (habilitadas).
- En bases de datos nativas XML.

2.1 Ficheros

No es la mejor opción, puesto que no puede garantizarse la concurrencia, integridad de atomicidad y el nivel de seguridad que ofrecen otros sistemas como las bases de datos.

2.2 Bases de datos XML enabled

En realidad, es una base de datos relacional que convierte documentos XML en un esquema relacional y, una vez hecho esto, vuelca los datos en dicho esquema. Estas bases de datos suelen ser bases de datos tradicionales. Permiten XML como entrada y pueden generar XML como salida.

La base de datos procesa directamente el XML sin necesidad de utilizar **middleware** adicional.

Presentan los siguientes problemas:

- No puede restaurarse el documento original.
- Diferencia de filosofía. XML es jerárquico, mientras las bases de datos XML *enabled* son relacionales.

Definición

El middleware es **un** software que **proporciona** servicios y **funcionalidades** comunes a las **aplicaciones más allá** de lo que **proporciona** el sistema operativo.

Existen numerosos SGBD que soportan XML habilitado.

- **IBM DB2:** Es un sistema de base de datos **básico** diseñado para **manejar** transacciones y **gestionar** consultas **complejas.**
  Ofrece funciones a escala **empresarial** y está **optimizado** para hasta 8 núcleos de **CPU** y 8 GB de **memoria.**

- **Oracle Database:** Es un **administrador** de **bases** de datos **relacionales** que **aprovecha** los recursos del sistema informático en todas las arquitecturas de hardware para garantizar **la máxima utilización** en **entornos ricos en información.**
  Es un conjunto de datos que se pueden almacenar y **acceder recursivamente utilizando** un modelo definido como relacional.

- **Microsoft SQL Server:** Es un Sistema para la gestión de Bases de Datos producido por Microsoft basado en el modelo relacional cuya misión principal es la de almacenar y consultar datos solicitados por otras aplicaciones, sin importar si están en la misma computadora, si están conectados a una red de área local o a Internet (plataforma " Cloud Ready").
- **PostgreSQL:** Es un SGBD relacional orientado a objetos y libre, publicado bajo la licencia BSD.
  Al igual que muchos otros proyectos de código abierto, el desarrollo de PostgreSQL no **está impulsado** por una empresa y/o **un individuo,** sino por una comunidad de desarrolladores **desinteresados, despreocupados, libres** y/o **respaldados** por organizaciones comerciales.

2.3 Bases de datos nativas

Su modelo interno se basa en XML y su unidad de almacenamiento es el documento XML (para las bases de datos relacionales es la fila). Son las que mejores prestaciones ofrecen con este tipo de documentos.

A diferencia de las bases de datos XML *enabled*, puede recuperarse el documento original, puesto que lo almacenan sin alterarlo.

Cuando los documentos son grandes o complejos, las bases de datos XML *enabled* suelen generar problemas y la mejor opción es utilizar una base de datos nativa.

Además, estas bases de datos utilizan el modelo jerárquico del documento XML mediante nodos y cada nodo tendrá sus comentarios, atributos, instrucciones, etc.

En estos entornos **XML** nativos ya no se utilizan lenguajes como SQL, sino **que** otros **lenguajes como XQuery y XPath se han adaptado** a este **modelo.**

Algunas de las bases de datos XML son:

- **MarkLogic**: Con la licencia propietaria.
- **BaseX**: Con licencia BSD.
- **eXist-db**: Con licencia LGPL.
- **Sedna**: Con licencia Apache.

3. Manipulación de información en documentos XML: inserción y extracción

Muchas herramientas son capaces de guardar y leer documentos en XML.

Esto es así porque XML es un formato universal y sencillo de generar y analizar (**parsear**).

Una herramienta muy interesante para cualquier programador web es **ImportXML** y se encuentra en la suite de **Google Docs**, específicamente en la hoja de cálculo.

Esta función permite importar datos de formatos estructurados como puede ser XML, HTML, CSV, etc.

La sintaxis de la fórmula es:

IMPORTXML (url;consulta_xpath)

- *url*: URL de la página que desea examinarse, hay que incluir el protocolo http://. Esta URL tiene que ir entre comillas o hacer referencia a una celda.
- *consulta_path*: es la consulta en lenguaje **XPath** que va a ejecutarse sobre la URL anterior.

Esta función puede servir para extraer cualquier información de una página web o cualquier otro formato estructurado de datos.

Más info

Para visualizar el uso de esta función en Google Sheets puedes visualizar el siguiente vídeo:

[Google Sheets] Funcion ImportXMLVer enlace

4. Técnicas de búsqueda de información en documentos XML

El lenguaje XML se concibió como un lenguaje para el intercambio de información en un formato estandarizado.

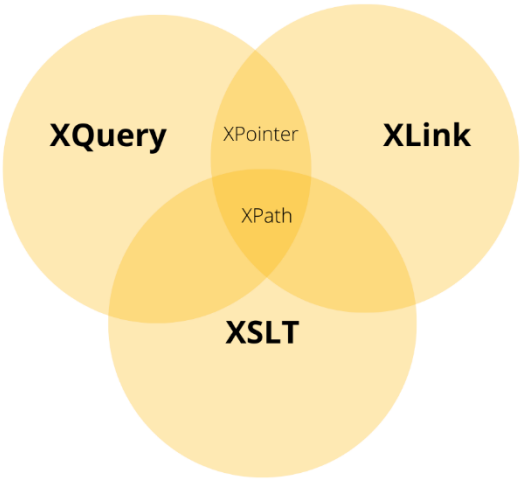
Esto es posible debido a que la estructura de los documentos XML es conocida y cumple ciertas reglas, como por ejemplo que la estructura es jerárquica o que los elementos tienen que estar contenidos de manera completa unos dentro de otros. Una de las operaciones más complejas de realizar sobre un documento XML es la búsqueda de información.

Mediante el uso de analizadores como **SAX** o **DOM** es posible realizar búsquedas de información, pero la forma de hacerlo requiere obligatoriamente conocimientos de programación. Además, suelen ser programas engorrosos de desarrollar y poco eficientes.

A partir de esta estructura conocida se han desarrollado una serie de tecnologías relacionadas con XML que automatizan tareas como la validación o la transformación.

También existen tecnologías que automatizan las tareas de búsqueda de información concreta dentro de los documentos XML: **XPath** y **Xquery** son algunas de ellas. Están muy relacionadas y se complementan.

XPath y **XQuery** proporcionan una herramienta similar a **SQL** en el acceso a los datos de las bases de datos relacionales.



XQuery + XLink + XSLT

5. Lenguaje de consulta y manipulación

Cuando se habla de lenguajes de consulta, generalmente se habla de **XPath** y **XQuery**.

La diferencia es que XPath es un lenguaje para seleccionar nodos en el lenguaje XML dentro de un DOM, mientras que XQuery, además de ser un subconjunto de XPath, ofrece sintaxis FLWOR.

FLWOR: For, Let, Where, Order by y Return

Esto lo convierte en un lenguaje más completo para consultas y transformaciones de datos.

5.1 XPath

Se trata de un lenguaje de ruta para XML. Utiliza una sintaxis *path like* para identificar y navegar entre los nodos de un documento XML. Dispone un gran número de funciones predefinidas, más de 200, entre ellas:

- Manipulación de cadenas (substring (), contains (), string-length ()).
- Operaciones booleanas y numéricas (=, <, >, not()).
- Comparación de fechas y horas (current-date(), current-time()).
- Manejo de secuencias y nodos (position (), last()).

Las expresiones XPath pueden utilizarse en Java, JavaScript, PHP, Python, C, C++, etc. XPath se utiliza, como ya hemos visto, en XSLT.

XPath utiliza un analizador cuya función es crear un árbol de nodos a partir de un documento XML.

Los nodos creados son de siete tipos: nodo raíz, elemento, texto, atributo, namespace (espacio de nombres), instrucción procesable y comentario.

Ejemplo 1 – Seleccionar todos los títulos de un catálogo de libros:

```
IMPORTXML(url; consulta_xpath)
<catalogo>
 <libro>
  <titulo>El Quijote</titulo>
  <autor>Cervantes</autor>
 </libro>
 <libro>
  <titulo>La Odisea</titulo>
  <autor>Homero</autor>
```

```
  </libro>
</catalogo>
```

Consulta XPath:

```
/catalogo/libro/titulo
```

Resultado:

```
El Quijote
La Odisea
```

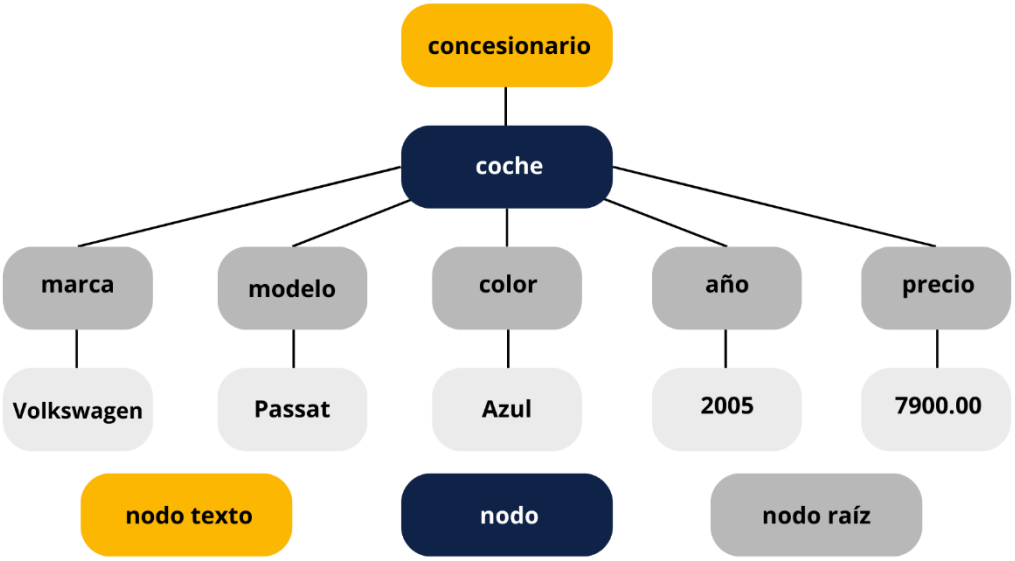
Ejemplo 2 – Seleccionar solo el primer libro:

```
/catalogo/libro[1]/titulo
```

Resultado:

```
El Quijote
```

5.1.1 Los nodos en XPath



Arbol de nodos creado por XPath

Como se puede ver en la figura anterior, los nodos formaran parte de la estructura arborescente creada por el parser de XPath. Los nodos forman la base de la jerarquía de un documento XML.

En él pueden encontrarse:

- **Nodo**: Un ejemplo de nodo seria coche, modelo, color, etc.
- **Nodo raíz**: Es el que se sitúa en la parte superior del árbol (concesionario).
- **Valor atómico**: En el ejemplo anterior, podría ser "Passat" o "2005".
- **Ítem o elemento**: Puede ser un nodo o un valor atómico.

Los nodos en las estructuras arborescentes de XML están relacionados unos con otros. Pueden encontrarse nodos:

- **Padre (parent):** Un nodo es padre de otro cuando este último desciende de él. Por ejemplo, coche desciende de concesionario.
- **Hijo (children):** El hijo es el nodo que desciende del nodo padre. Por ejemplo, coche es nodo hijo de concesionario.
- **Hermanos (siblings):** Son aquellos nodos hijo del mismo padre (marca, modelo, color, etc.)

- **Antecedores (ancestors):** Los nodos coche y concesionario serán antecesores de color, puesto que jerárquicamente están en un orden superior directo.
- **Descendientes (descendant):** Son los hijos de los hijos, etc. de un nodo.

Ejemplo:

```
<concesionario>
 <coche>
  <marca>Volkswagen</marca>
  <modelo>Passat</modelo>
  <año>2005</año>
  <color>Negro</color>
 </coche>
</concesionario>
```

- Concesionario → Nodo raíz.
- Coche → Hijo de concesionario.
- Marca, modelo, año, color → Hijos de coche y hermanos entre sí.
- Passat, 2005, Negro → Valores atómicos.
- Coche y concesionario → Antecesores de color.

5.1.2 Como seleccionar los nodos de un documento

Para seleccionar los nodos de un documento XML, **XPath** utiliza expresiones y, de esa manera, un nodo puede ser seleccionado siguiendo una serie de pasos o ruta.

Expresiones utilizadas por XPath en la selección de nodos:

Expresión	Expresión XPath	Ejemplo sobre XML	Resultados
nombre_nodo	Selecciona todos los nodos con el nombre *nombre_nodo*	//coche	Devuelve todos los nodos <coche>
/	Selecciona desde el nodo raíz	/concesionario	Devuelve el nodo raíz <concesionario>
//	Selecciona nodos del	//marca	Devuelve todos los nodos <marca>

	documento que cumplan los criterios de selección desde el nodo actual sin importar donde se encuentren		
.	Selecciona el nodo actual	.	Devuelve el nodo en el que te encuentras
..	Selecciona el nodo padre del nodo actual	../marca	Devuelve el nodo <marca> padre del nodo actual
@	Selecciona los atributos	//coche/@id	Devuelve el valor del atributo id de cada coche

Ejemplo de XML (con predicados incluidos):

```
<concesionario>
 <coche id="1">
  <marca>Volkswagen</marca>
  <modelo>Passat</modelo>
  <año>2005</año>
  <color>Negro</color>
 </coche>
 <coche id="2">
  <marca>Audi</marca>
  <modelo>A4</modelo>
  <año>2010</año>
  <color>Blanco</color>
 </coche>
 <coche id="3">
  <marca>Volkswagen</marca>
  <modelo>Golf</modelo>
  <año>2018</año>
  <color>Rojo</color>
 </coche>
</concesionario>
```

Ejemplos de uso de predicados en XPath:

Expresión XPath	Resultado
/concesionario/coche[1]	Devuelve el primer coche (Passat).
/concesionario/coche[last()]	Devuelve el último coche (Golf).
/concesionario/coche[@id="2"]	Devuelve el coche con id=2 (Audi A4).
/concesionario/coche[marcha="Volkswagen"]	Devuelve los coches cuya marca sea Volkswagen (Passat y Golf).
/concesionario/coche[color="Rojo"]/modelo	Devuelve el modelo del coche rojo (Golf).
//coche[año>2008]	Devuelve los coches cuyo año sea mayor que 2008 (Audi A4 y Golf).

Con esto, queda claro cómo usar XPath para navegar y seleccionar nodos específicos, tanto con expresiones básicas como con predicados más avanzados.

5.1.3 Utilizando predicados para mejorar la búsqueda

En **XPath**, un predicado es una expresión que se utiliza para filtrar un conjunto de nodos basándose en ciertas condiciones. Los predicados se utilizan para seleccionar subconjuntos específicos de nodos en un documento XML y permiten realizar consultas más precisas y específicas.

Un predicado se agrega a una expresión de ruta XPath colocando una expresión entre corchetes [ ]. Esta expresión puede incluir operadores de comparación, funciones XPath y otros elementos de lenguaje XPath que permiten especificar las condiciones de filtrado.

Ejemplos prácticos de predicados en XPath:

Suponiendo este fragmento de XML:

```
<concesionario>
 <coche>
  <marca>Volkswagen</marca>
  <modelo>Passat</modelo>
  <precio>9000.00</precio>
```

```
 </coche>
 <coche>
  <marca>Opel</marca>
  <modelo>Astra</modelo>
  <precio>8500.00</precio>
 </coche>
 <coche>
  <marca>Seat</marca>
  <modelo>Ibiza</modelo>
  <precio>7500.00</precio>
 </coche>
 <coche>
  <marca>Renault</marca>
  <modelo>Clio</modelo>
  <precio>6500.00</precio>
 </coche>
</concesionario>
```

Expresión	Resultado
`/concesionario/coche[1]/marca`	Seleccionaría la marca del primer coche del concesionario. En algunos navegadores, los nodos empiezan a contar desde el 0.
`/concesionario/coche[last()]/marca`	Seleccionaría la marca del ultimo coche del concesionario.
`/concesionario/coche[position()<4]/marca`	Seleccionaría la marca de los tres primeros coches del concesionario.
`/concesionario/coche[precio>8000.00]/modelo`	Selecciona el modelo de aquellos coches cuyo precio sea superior a 8000 euros. El resultado es el siguiente: Modelo: Touran Modelo: Astra

Ejemplos de predicados XPath

Con los predicados, XPath pasa de ser un simple recorrido de nodos a una herramienta de **consulta avanzada**, casi como un "mini-SQL para XML".

5.1.4 Cambiar varias rutas

En algunas situaciones no se desea seleccionar un único nodo, sino varios nodos con distinto contenido dentro de un mismo documento XML.

Para ello, en XPath se utiliza el operador |, que actúa como un OR lógico y permite unir distintas rutas en una misma expresión.

Ejemplo:

Supongamos el siguiente fragmento XML de un concesionario:

```
<concesionario>
 <coche>
  <modelo>Passat</modelo>
  <precio>7900.00</precio>
 </coche>
 <coche>
  <modelo>Touran</modelo>
  <precio>8200.00</precio>
 </coche>
 <coche>
  <modelo>Astra</modelo>
  <precio>8490.00</precio>
 </coche>
</concesionario>
```

Si queremos obtener **el modelo y el precio de todos los coches**, la ruta XPath sería:

```
/concesionario/coche/modelo | /concesionario/coche/precio
```

Resultado esperado:

```
Passat
Touran
Astra
7900.00
8200.00
8490.00
```

El motivo por el cual aparecen primero los modelos y después los precios es que **XPath selecciona los nodos siguiendo el orden definido en la expresión path.**

En este caso, primero se pidió modelo y luego precio, por lo que respeta ese orden en el resultado.

5.1.5 Comodines

En **XPath**, los comodines son caracteres especiales que se utilizan para representar cero o más caracteres en una expresión de ruta. Los comodines permiten hacer coincidir patrones más amplios en lugar de especificar exactamente un valor específico.

Comodín	Descripción
*	Selecciona cualquier nodo elemento.
@*	Selecciona cualquier nodo atributo.
node()	Selecciona cualquier nodo de cualquier clase.
?	Selecciona un solo carácter.

Dado el siguiente XML:

```
<concesionario>
 <coche>
  <marca>Volkswagen</marca>
  <modelo>Passat</modelo>
  <color>Azul</color>
  <año>2005</año>
  <precio>7900.00</precio>
 </coche>
 <coche>
  <marca>Volkswagen</marca>
  <modelo>Touran</modelo>
  <color>Azul</color>
  <año>2007</año>
```

```
  <precio>8200.00</precio>
 </coche>
 <coche>
  <marca>Opel</marca>
  <modelo>Astra</modelo>
  <color>Negro</color>
  <año>2013</año>
  <precio>8490.00</precio>
 </coche>
</concesionario>
```

Si utilizamos la ruta:

```
/concesionario/coche/*
```

El resultado sería **todos los elementos hijos de cada coche**, sin necesidad de especificarlos uno por uno:

```
Volkswagen
Passat
Azul
2005
7900.00
Volkswagen
Touran
Azul
2007
8200.00
Opel
Astra
Negro
2013
8490.00
```

El comodín * es especialmente útil cuando no conocemos de antemano el nombre de todos los nodos, o cuando queremos **extraer toda la información de un bloque** sin necesidad de escribir cada ruta explícitamente.

Más info

Para ampliar información sobre XPath

visualiza el siguiente vídeo: Expresiones XPath Absolutas y Relativas

5.2 XQuery

XQuery es un lenguaje de consultas diseñado para extraer y manipular información almacenada en documentos XML. Incorpora todo XPath, pero añade la poderosa sintaxis **FLWOR**:

- **F**or → Iterar sobre nodos.
- **L**et → Definir variables temporales.
- **W**here → Aplicar condiciones.
- **O**rder by → Ordenar resultados.
- **R**eturn → Devolver resultados.

Ejemplo 1 – Obtener los títulos de todos los libros:

```
for $libro in /catalogo/libro
return $libro/titulo
```

Resultado:

```
El Quijote
La Odisea
```

Ejemplo 2 – Seleccionar solo libros de un autor específico:

```
for $libro in /catalogo/libro
where $libro/autor = "Homero"
return $libro/titulo
```

Resultado:

```
La Odisea
```

Ejemplo 3 – Ordenar libros por título:

```
for $libro in /catalogo/libro
order by $libro/titulo
return concat($libro/titulo, " - ", $libro/autor)
```

Resultado:

```
El Quijote - Cervantes
La Odisea - Homero
```

De esta forma queda claro:

- XPath sirve para **consultas directas** (extraer nodos).
- XQuery permite consultas más **complejas y estructuradas** (filtrado, ordenación, composición de resultados).

XQuery es un lenguaje de consulta diseñado para recuperar y manipular datos de documentos XML. Fue desarrollado por el grupo de trabajo XML Query del Consorcio World Wide Web (**W3C**) y es un estándar W3C desde enero de 2007.

XQuery se utiliza para realizar consultas en documentos XML, el SQL tiene una función similar, ya que se usa para consultar bases de datos relacionales. Las consultas XQuery pueden acceder a los datos de un documento XML, filtrar y ordenar los resultados y devolver los resultados en varios formatos de salida, como XML, HTML, texto plano, JSON y otros formatos de archivo.

Algunas de las características de XQuery incluyen:

- Sintaxis familiar para quienes ya conocen SQL: Esto facilita el aprendizaje para programadores y administradores de bases de datos.
- Acceso a múltiples documentos XML en una misma consulta: Se pueden unir y combinar datos de distintos orígenes.
- Soporte de expresiones complejas y funciones integradas: Más de 200 funciones para trabajar con cadenas, números, fechas, booleanos, nodos y secuencias.
- Capacidad de transformación: XQuery no solo recupera datos: puede transformarlos y devolver un resultado nuevo.
- Independencia del lenguaje de programación: XQuery se puede usar en aplicaciones Java, .NET, Python, PHP, etc.

- Integración con XPath: XQuery incluye XPath como sublenguaje, por lo que toda expresión XPath es válida dentro de XQuery.

Ejemplo básico de XQuery

Imaginemos que tenemos un documento XML con coches:

```
<concesionario>
 <coche>
  <marca>Volkswagen</marca>
  <modelo>Passat</modelo>
  <precio>22000</precio>
 </coche>
 <coche>
  <marca>Seat</marca>
  <modelo>León</modelo>
  <precio>18000</precio>
 </coche>
 <coche>
  <marca>Audi</marca>
  <modelo>A3</modelo>
  <precio>28000</precio>
 </coche>
</concesionario>
```

Consulta en XQuery:

```
for $x in doc("coches.xml")/concesionario/coche
where $x/precio > 20000
order by $x/modelo
return <li>{data($x/modelo)}</li>
```

Explicación:

- for → recorre cada coche del concesionario.
- where → filtra los coches cuyo precio es mayor de 20.000.
- order by → ordena los resultados por el modelo.
- return → devuelve el modelo dentro de un elemento <li>.

Salida:

```
<li>A3</li>
```

```
<li>Passat</li>
```

Las expresiones que se utilizan en XQuery se denominan sentencias **FLWOR** (For, Let, Where, Order by y Return).

FLWOR es un acrónimo de "For, Let, Where, Order by, Return", que se refiere a una construcción de lenguaje en XQuery que se utiliza para realizar consultas y transformaciones de datos en documentos XML. FLWOR es una forma poderosa y flexible de expresar consultas en XQuery, ya que permite combinar varias operaciones y procesar datos complejos de manera eficiente.

Cada letra en el acrónimo FLWOR representa una cláusula de lenguaje que se utiliza en la construcción FLWOR:

- **For**: La cláusula "For" se utiliza para crear variables que representan nodos o secuencias de nodos en el documento XML. Estas variables pueden ser utilizadas en otras cláusulas de la construcción FLWOR.
- **Let**: La cláusula "Let" se utiliza para crear variables que representan valores o expresiones en la consulta. Estas variables también pueden ser utilizadas en otras cláusulas de la construcción FLWOR.
- **Where**: La cláusula "Where" se utiliza para filtrar los resultados de la consulta utilizando una expresión booleana. Solo se devuelven los resultados que cumplan la condición especificada en la cláusula "Where".
- **Order by: Ordenar por:** la cláusula **"Ordenar por"** se utiliza para ordenar los resultados de la consulta en base a una o más claves especificadas en dicha cláusula. Los resultados pueden ordenarse de forma ascendente o descendente, según se especifique en la cláusula
- **Return**: La cláusula "Return" se utiliza para especificar los resultados que se devolverán en la consulta. Puede incluir una expresión que combine las variables y las funciones de XQuery para producir el resultado deseado.

La construcción FLWOR en XQuery es extremadamente flexible y se puede utilizar para crear consultas y transformaciones de datos complejas y sofisticadas en documentos XML.

Ejemplo:

Documento XML de libros:

```
<library>
 <book>
  <title>El gran Gatsby</title>
```

```
  <author>F. Scott Fitzgerald</author>
  <year>1925</year>
 </book>
 <book>
  <title>Cien años de soledad</title>
  <author>Gabriel García Márquez</author>
  <year>1967</year>
 </book>
 <book>
  <title>1984</title>
  <author>George Orwell</author>
  <year>1949</year>
 </book>
</library>
```

Consulta en XQuery:

```
for $book in //book
where $book/year ≤ 1960
order by $book/title
return
 <book>
  <title>{ $book/title/text() }</title>
  <author>{ $book/author/text() }</author>
  <year>{ $book/year/text() }</year>
 </book>
```

Explicación:

- for crea una variable $book para cada libro.
- where filtra libros publicados antes de 1960.
- order by ordena los libros por título.
- return devuelve un nuevo XML con solo los libros seleccionados.

Salida:

```
<book>
 <title>1984</title>
 <author>George Orwell</author>
 <year>1949</year>
```

```
</book>
<book>
  <title>El gran Gatsby</title>
  <author>F. Scott Fitzgerald</author>
  <year>1925</year>
</book>
```

Ejemplo con let:

Supongamos que queremos calcular el precio medio de los coches:

```
let $precios := doc("coches.xml")//coche/precio
return avg($precios)
```

Ejemplo con varias condiciones:

```
for $x in //coche
where $x/precio > 20000 and $x/marca = "Audi"
return $x/modelo
```

Salida:

```
A3
```

Usos más comunes de XQuery:

- Transformar datos XML a XHTML o PDF.
- Realizar búsquedas complejas en documentos XML.
- Generar informes automáticos con datos de XML.
- Mostrar información en servicios web.
- Procesar ficheros XML grandes de forma eficiente.
- Integración empresarial, permitiendo a distintas aplicaciones intercambiar datos en XML.

Diferencia entre XPath y XQuery:

XPath	XQuery
Lenguaje para **seleccionar nodos** en documentos XML.	Lenguaje completo de **consulta y transformación**.
Sintaxis simple (rutas y filtros).	Sintaxis más rica (FLWOR), variables, funciones).
Devuelve valores directos (ej: el título de un libro).	Devuelve estructuras completas (puede generar nuevo XML, HTML, JSON).
Ideal para tareas puntuales o dentro de XSLT.	Ideal para aplicaciones más complejas o informes.

5.2.1 Aplicación para trabajar con XQuery

BaseX es un sistema de gestión de bases de datos XML de código abierto y alto rendimiento. Ofrece una amplia gama de funciones para la creación, consulta y manipulación de bases de datos XML y es compatible con una variedad de estándares XML y tecnologías relacionadas.

Entre las características principales de BaseX se incluyen:

- Almacenamiento y recuperación eficiente de grandes volúmenes de datos XML.
- Búsqueda de texto completo y búsqueda de texto parcial, incluyendo soporte para consultas de texto y de estructura.
- Análisis sintáctico y validación de documentos XML según diferentes esquemas XML.
- Transformaciones XSLT y XQuery de alto rendimiento.
- Soporte para XPath 2.0, XQuery 3.1 y XQuery Update.
- Soporte para múltiples usuarios y seguridad basada en roles.
- API y lenguajes de programación compatibles, incluyendo Java, Python y PHP.

BaseX es fácil de instalar y usar. Se puede ejecutar en diferentes sistemas operativos, incluyendo Windows, Mac OS X y Linux. Además, se puede integrar con otras tecnologías y sistemas de gestión de bases de datos, lo que lo hace una opción ideal para aplicaciones de gestión de contenido y aplicaciones web que requieren una gestión de datos XML eficiente.

Más info

Puedes descargarte la aplicación en el siguiente enlace:

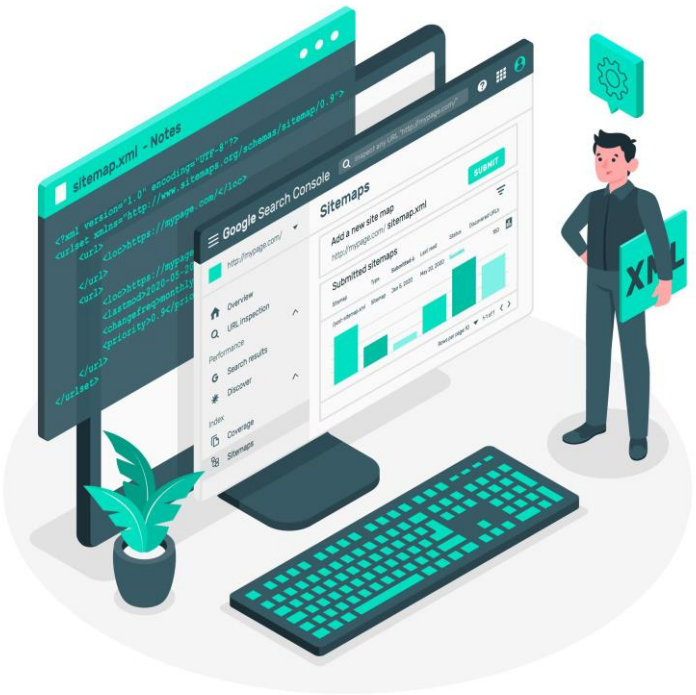
https://basex.org/download/

Para ampliar información sobre la instalación de BaseX y la utilización de Xquey visualiza el vídeo

XQuery de XML con BaseX

6. Almacenamiento XML nativo

El **almacenamiento XML nativo** es un enfoque de almacenamiento de datos en el que la información se almacena directamente en formato XML en una base de datos o sistema de gestión de información. En otras palabras, en un sistema de almacenamiento XML nativo, los datos se almacenan en su forma original de XML, sin necesidad de convertirlos a otro formato.



Este enfoque tiene varias ventajas, como una mayor flexibilidad para la organización y el procesamiento de datos, una mayor facilidad para realizar consultas complejas y la capacidad de mantener la integridad de los datos y su estructura original. Además, el almacenamiento de datos en formato XML nativo es particularmente útil para aplicaciones que trabajan con grandes volúmenes de información no estructurada y para entornos en los que la integración y el intercambio de datos son fundamentales.

Sin embargo, el almacenamiento XML nativo también puede presentar algunos desafíos, como un mayor tamaño de almacenamiento y una posible pérdida de rendimiento en comparación con otros formatos de almacenamiento de datos. Por esta razón, la elección de un enfoque de almacenamiento depende de los requisitos específicos de la aplicación y de las características de la información que se va a almacenar.

6.1. Instalación y ejecución de una base de datos XML nativa en eXist-db

Se ha elegido **eXist-db** por ser una base de datos de código abierto y multiplataforma. Estas características hacen que sea una de las bases de datos No-SQL más difundidas.

Instalar eXistdb es muy fácil, pero para poder hacerlo primero debemos cumplir con ciertos requisitos, en primer lugar, tener un sistema operativo Windows, MAC OS o Linux.

En segundo lugar, debemos tener Java 1.7 o posterior. Generalmente las pruebas que el equipo de eXistdb hace son sobre el entorno oficial de Oracle y también sobre el entorno libre OpenJDK, por lo que podemos esperar compatibilidad máxima en la mayoría de los entornos en los que nos manejemos.

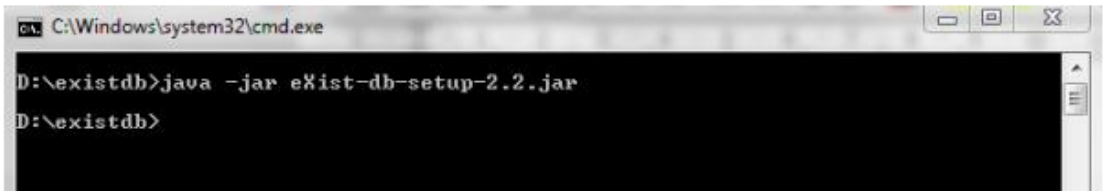
Una vez que hayamos cumplido los requisitos mencionados anteriormente, el siguiente paso es visitar el sitio web exist-db.org y dirigirnos a la sección de descargas, donde encontraremos la última versión disponible del software.

Al hacer clic en la versión más reciente, se descargará un archivo .jar que contiene todos los recursos necesarios para la instalación de eXistdb en nuestro sistema. Una vez que hayamos obtenido el archivo .jar, debemos seleccionar el directorio en el que deseamos instalar el software y colocar allí el archivo descargado.

Luego, abrimos la terminal o consola de comandos en nuestro sistema y escribimos las siguientes instrucciones para continuar con el proceso de instalación:

```
java -jar exist-db-setup-xxx.jar
```

Donde las xxx son el número de la versión y revisión que hemos descargado, veamos el ejemplo en la siguiente imagen para mayor claridad:



Instrucción en Símbolo de Sistema en Windows

Esto nos va a levantar una interfaz gráfica que contendrá todos los parámetros para poder configurar **eXistdb** en nuestro sistema.

El primer paso, luego de empezar a instalar es elegir el directorio de instalación. Si hemos seguido este tutorial ya lo habremos creado o elegido adecuadamente, es muy importante no instalar en una ubicación como " Program Files" o " Archivos de Programa" en ambientes Windows ya que si no tendremos un conflicto con los permisos del sistema operativo.

Acto seguido, debemos elegir la carpeta donde se debe almacenar los datos del motor de Base de Datos, es decir, es allí dónde irán a parar nuestros documentos una vez empecemos a utilizar la aplicación, normalmente se escoge para ambientes de desarrollo la misma ruta de la instalación, que justamente es la propuesta por el instalador.

En este cuarto paso, debemos elegir en primer lugar la contraseña del administrador, por lo que debemos tomar nota y construir un password seguro. Hay casos como entornos de desarrollo donde podemos ser un poco más descuidados con esta contraseña, sin embargo, siempre es bueno utilizar contraseñas seguras.

Después de haber cumplido con los requisitos mencionados anteriormente, simplemente debemos acceder al sitio exist-db.org y dirigirnos a la sección de descargas, donde encontraremos el enlace de la última versión disponible que podemos seleccionar.

A continuación, se nos solicita configurar el almacenamiento para el servicio, y podemos seguir estas instrucciones:

- **Entre 512 MB de memoria y 64 MB de caché:** Este es el requisito mínimo para ejecutar el servicio. Un valor inferior puede ocasionar problemas, como el mal funcionamiento o la incapacidad de iniciarse.
- **Entre 1024 MB de memoria y 128 MB de caché:** Este es el valor predeterminado y se recomienda para máquinas de desarrollo o servidores de datos pequeños. Es un punto de partida adecuado si eres nuevo en eXistdb.
- **2048 MB de memoria o más y 256 MB de caché o más:** Si tu sistema lo permite, puedes utilizar esta configuración para obtener el mejor rendimiento posible al instalar nuestro servicio.

A continuación, debemos seleccionar los paquetes de instalación, y por simplicidad, optaremos por los predeterminados.

Una vez realizado este paso, eXistdb comenzará a copiar todos los archivos necesarios en nuestro sistema para que funcione correctamente.

Es muy importante que no haya otra aplicación utilizando estos puertos ya que puede haber un conflicto y estropearse ambas, por ello antes de levantar el servicio de eXistdb debemos verificar navegando a localhost:8080 y localhost:8443 y ver que no tengamos nada en el navegador. Aunque nuestro conocimiento del sistema operativo nos permita hacerlo, podemos verificar en el panel de control los puertos utilizados para confirmar que los dos puertos que hemos indicado no están en uso.

Para iniciar el servicio, simplemente debemos ejecutar la siguiente instrucción en el directorio de instalación:

```
java -jar start.jar
```

Inmediatamente obtendremos una pantalla indicándonos el inicio de eXistdb.

Más info

Para ampliar información sobre la instalación de eXistdb visualiza el vídeo eXistdb Instalacion.

7. Herramientas de tratamiento y almacenamiento de información en formato XML

Actualmente, existen infinidad de herramientas con las cuales poder editar un documento XML.

A diferencia de un editor de texto simple como Notepad o Gedit, los editores especializados en XML ofrecen múltiples ventajas:

- Identifican y señalan errores de sintaxis en tiempo real.
- Formatean automáticamente el código, aplicando sangrías correctas.
- Cierran etiquetas de manera automática para evitar fallos de estructura.
- Mejoran la legibilidad y la productividad del programador.

Estas herramientas ya se han ido presentando y utilizando a lo largo de las cinco unidades didácticas anteriores, por lo que en este punto el estudiante debería estar familiarizado con su funcionamiento y utilidad.

Tech Hub <FP> euroformac

8. Resumen

XML es una tecnología versátil que engloba numerosos conceptos como XML básico, XML Namespaces, XML Schema, XSLT, XPath y XQuery, entre otros.

El uso de bases de datos XML está en crecimiento, especialmente en entornos donde el ancho de banda es limitado o donde el rendimiento y la eficiencia resultan críticos. Gracias a su estructura jerárquica y su flexibilidad, XML ha demostrado ser un estándar efectivo y fiable para el almacenamiento y el intercambio de información.

La información contenida en documentos XML puede consultarse y manipularse mediante lenguajes especializados como XPath y XQuery, que permiten realizar búsquedas, filtrados y transformaciones de manera precisa.

Por último, las bases de datos nativas XML representan la mejor opción cuando se requiere trabajar directamente con documentos XML en su forma original, ya que preservan íntegramente su estructura jerárquica y ofrecen un alto rendimiento en la gestión de datos complejos.

Glosario

- **XSLT:** XSLT, o Transformaciones XSL, es un estándar desarrollado por la organización W3C que proporciona una metodología para convertir documentos XML en otros formatos, incluso aquellos que no son XML. Las hojas de estilo XSLT se utilizan para llevar a cabo la transformación del documento mediante la aplicación de una o varias reglas de plantilla vinculadas al documento fuente que se desea transformar.

- **DOM:** El Modelo de Objetos del Documento (DOM), también conocido como "Modelo en Objetos para la Representación de Documentos", es una interfaz esencial en las plataformas que proporciona un conjunto estándar de objetos para representar documentos HTML, XHTML y XML. Este modelo establece una forma estándar de combinar estos objetos, así como una interfaz común para acceder y manipularlos. El objetivo principal del diseño del DOM es permitir a los programas acceder y modificar el contenido, la estructura y el estilo de los documentos HTML y XML. Esto significa que el DOM brinda la capacidad de interactuar con los elementos y propiedades de los documentos de una manera programática.

- **Xpath:** XPath (XML Path Language) es un lenguaje que posibilita la construcción de expresiones para recorrer y procesar documentos XML. XPath opera de manera similar a las expresiones regulares utilizadas para seleccionar partes de un texto sin atributos (texto plano). Sin embargo, XPath va más allá al permitir la búsqueda y selección teniendo en cuenta la estructura jerárquica del XML. Esto significa que XPath permite navegar y filtrar elementos XML en función de su posición y relación con otros elementos en el documento. Fue diseñado específicamente para ser utilizado en el estándar XSLT, donde se emplea para seleccionar y examinar la estructura del documento de entrada durante la transformación. XPath fue definido por el consorcio W3C.

- **XQuery**: XQuery es un lenguaje de consulta diseñado específicamente para manejar colecciones de datos XML. Aunque comparte cierta similitud semántica con SQL, XQuery también incorpora capacidades de programación adicionales.

- **Java**: Java es un lenguaje de programación orientado a objetos de alto nivel que fue creado por Sun Microsystems en la década de 1990 y actualmente es propiedad de Oracle Corporation. Se ha convertido en uno de los lenguajes de programación más populares y ampliamente

utilizados en todo el mundo, tanto en la industria como en el ámbito educativo.

Java se caracteriza por su portabilidad, es decir, la capacidad de ejecutarse en diferentes sistemas operativos y arquitecturas de hardware sin necesidad de realizar cambios en el código fuente. Esto se logra gracias a la implementación de una máquina virtual Java (JVM) que se encarga de interpretar y ejecutar el código Java compilado.

Adicionalmente, Java ofrece una extensa gama de bibliotecas y frameworks que agilizan el desarrollo de aplicaciones de diversas categorías. Estas abarcan desde aplicaciones web y móviles, hasta sistemas de gestión empresarial y aplicaciones de inteligencia artificial.

En resumen, Java es un lenguaje de programación popular y ampliamente utilizado que se caracteriza por su portabilidad y su amplia gama de bibliotecas y frameworks.

- **Phyton**: Python es un lenguaje de programación de alto nivel, interpretado y orientado a objetos, creado en la década de 1990 por el programador holandés Guido van Rossum. Debido a su simplicidad, flexibilidad y facilidad de uso, Python se ha convertido en uno de los lenguajes de programación más populares y ampliamente utilizados en todo el mundo.
- **BaseX:** Sistema de gestión de bases de datos XML de código abierto y alto rendimiento. Soporta XQuery, XPath y XSLT, y permite almacenar y manipular grandes volúmenes de datos XML de manera eficiente. Está optimizado para consultas rápidas y soporta múltiples usuarios con gestión de seguridad.
- **eXist-db:** Base de datos nativa XML de código abierto y multiplataforma. Permite almacenar documentos XML en su forma original y realizar consultas avanzadas mediante XQuery y XPath. Es una de las bases de datos NoSQL más utilizadas en proyectos académicos y empresariales.
- **Almacenamiento XML nativo:** Método de almacenamiento en el que los datos se guardan directamente en formato XML, preservando su estructura jerárquica original. Facilita consultas complejas y mantiene la integridad del documento, aunque puede implicar mayor tamaño de almacenamiento.

- **XML Enabled Databases:** Bases de datos relacionales tradicionales que permiten importar y exportar datos en formato XML, transformando la jerarquía en tablas relacionales. Aunque útiles, no preservan siempre la estructura original del documento.

Tech Hub <FP> euroformac

Bibliografía

- Zurdo, S. F. J. T. R. & RA-MA Editorial. (2023). *LENGUAJES DE MARCAS Y SISTEMAS DE GESTIÓN DE LA INFORMACIÓN. CFGS.* (1.ª ed.). RA-MA Editorial.
- Castro, J. M., & Rodríguez, J. R. (2001). *Lenguajes de marcas y sistemas de gestión de la información*. Garceta.
- *XPath: XML Path language. XML. Bartolomé Sintes Marco. www.mclibre.org.* (s. f.). https://www.mclibre.org/consultar/xml/lecciones/xml-xpath.html
- *XQuery Tutorial.* (s. f.). http://www.w3bai.com/es/xsl/xquery_intro.html